AI ASSISTENT CODING 13.4

Baisa Vaishnavi
2303A52142
Batch - 41

Task 1: Refactoring Data Transformation

Logic

Scenario

You are maintaining a data preprocessing script where numerical transformations are written using verbose loops.

Task Description

• Review the legacy code that computes transformed values using an explicit loop.

• Use an AI tool to suggest a more Pythonic refactoring approach.

• Refactor the code using list comprehensions or helper functions while preserving the output.

Code:

13.1.py

C: > Users > Hello > 13.1.py > ...

```python
#Review the given Python legacy code that uses an explicit loop to transform a list of numerical val
#Ensure the final output remains [4, 8, 12, 16, 20].

# ========== ORIGINAL LEGACY CODE ==========
# values = [2, 4, 6, 8, 10]
# doubled = []
# for v in values:
#     doubled.append(v * 2)
# print(doubled)

# ========== REFACTORED APPROACH 1: LIST COMPREHENSION (RECOMMENDED) ==========
# Most Pythonic, concise, and performant
values = [2, 4, 6, 8, 10]
doubled = [v * 2 for v in values]
print("Approach 1 - List Comprehension:", doubled)

# ========== REFACTORED APPROACH 2: MAP WITH LAMBDA ==========
# Functional programming style
values = [2, 4, 6, 8, 10]
doubled = list(map(lambda v: v * 2, values))
print("Approach 2 - Map with Lambda:", doubled)

```

```python
# ========== REFACTORED APPROACH 3: MAP WITH HELPER FUNCTION ==========
# When logic needs to be reusable or complex
def double_value(v):
    """Helper function to double a numerical value"""
    return v * 2

values = [2, 4, 6, 8, 10]
doubled = list(map(double_value, values))
print("Approach 3 - Map with Helper Function:", doubled)

# ========== VERIFICATION ==========
# All approaches produce the expected output: [4, 8, 12, 16, 20]
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Hello\AppData\Local\Python\pythoncore-
\python.exe c:/Users/Hello/13.1.py
Approach 1 - List Comprehension: [4, 8, 12, 16, 20]
Approach 2 - Map with Lambda: [4, 8, 12, 16, 20]
Approach 3 - Map with Helper Function: [4, 8, 12, 16, 20]
PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code>
```

Justification:
The original code uses a verbose loop to perform a simple numerical transformation, which reduces readability. Refactoring it using a list comprehension makes the code more concise, Pythonic, and easier to understand while producing the same output. This approach also reduces the number of lines and improves maintainability.

Task 2: Improving Text Processing Code
Readability
Scenario
You are working on a log message generator that builds messages word by word.
Task Description
• Analyze the legacy code that constructs a sentence using repeated string concatenation.
• Ask AI to suggest a more efficient and readable approach.
• Refactor the code accordingly while keeping the final output Unchanged.

Code:

```
C: > Users > Hello > 13.2.py > ...
1   #Analyze the given Python legacy code that constructs a sentence using repeated string concatenation
2   # Original legacy code
3   sentence = ""
4   words = ["Refactoring", "with", "AI", "improves", "quality"]
5   for word in words:
6       sentence += word + " "
7   sentence = sentence.strip()
8   print(sentence)
9   # Refactored code using built-in string methods
10  words = ["Refactoring", "with", "AI", "improves", "quality"]
11  sentence = " ".join(words)
12  print(sentence)
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Hello\AppData\Local\Pyth
\python.exe c:/Users/Hello/13.2.py
Refactoring with AI improves quality
Refactoring with AI improves quality
PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code>
```

Justification:

The legacy code builds a string using repeated concatenation inside a loop, which is inefficient and harder to read. Using Python's built-in string joining methods results in cleaner, faster, and more readable code. The refactored solution preserves the exact output while following Python best practices.

Task 3: Safer Access to Configuration Data

Scenario

You are maintaining a configuration loader where dictionary keys may or may not exist depending on deployment settings.

Task Description

• Review the legacy code that manually checks for dictionary keys.

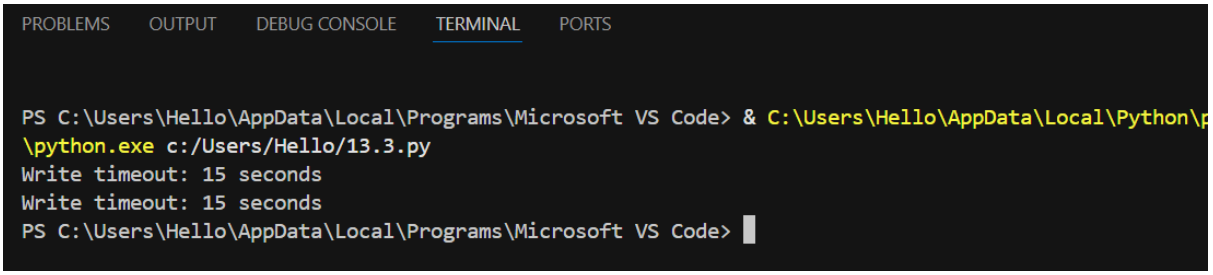
• Use AI suggestions to refactor the code using safer dictionary access methods.

• Ensure the behavior remains the same for missing keys.

Code:

```
C: > Users > Hello > 13.3.py > ...
1   #Review the given Python legacy code that manually checks for the existence of dictionary keys before
2   # Original legacy code
3   timeout_settings = {
4       "connection_timeout": 30,
5       "read_timeout": 60
6   }
7   if "write_timeout" in timeout_settings:
8       write_timeout = timeout_settings["write_timeout"]
9   else:
10      write_timeout = 15
11  print(f"Write timeout: {write_timeout} seconds")
12  # Refactored code using the get method
13  timeout_settings = {
14      "connection_timeout": 30,
15      "read_timeout": 60
16  }
17  write_timeout = timeout_settings.get("write_timeout", 15)
18  print(f"Write timeout: {write_timeout} seconds")
```

Output:



Justification:

The original code explicitly checks for the presence of a key before accessing it, which increases verbosity. Refactoring using safer dictionary access methods like `get()` simplifies the code, improves readability, and avoids potential key access errors while preserving the same behavior for missing configuration values.

Task 4: Refactoring Conditional Logic for Scalability

Scenario

You are enhancing a utility module that performs different operations based on user input.

Task Description

• Examine the multiple if–elif conditions used to determine operations.

• Ask AI to suggest a cleaner, scalable alternative.

• Refactor the logic using mapping techniques while preserving Functionality.

Code:

C: > Users > Hello > 13.4.py > ...

```python
#nalyze the given Python code that uses multiple if-elif conditions to perform operations based on
# Original legacy code
def perform_operation(operation, num1, num2):
    if operation == 'add':
        return num1 + num2
    elif operation == 'subtract':
        return num1 - num2
    elif operation == 'multiply':
        return num1 * num2
    elif operation == 'divide':
        return num1 / num2
    else:
        raise ValueError("Invalid operation")
# Refactored code using a dictionary for mapping operations
def add(num1, num2):
    return num1 + num2
def subtract(num1, num2):
    return num1 - num2
def multiply(num1, num2):
    return num1 * num2
def divide(num1, num2):
    return num1 / num2
operation_mapping = {
    'add': add,
    'subtract': subtract,
    'multiply': multiply,
    'divide': divide
}
```

```python
def perform_operation(operation, num1, num2):
    if operation in operation_mapping:
        return operation_mapping[operation](num1, num2)
    else:
        raise ValueError("Invalid operation")
# Example usage
result = perform_operation('divide', 10, 2)
print(result)
# Output
5.0
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Hello\AppData\Local\Py
\python.exe c:/Users/Hello/13.4.py
5.0
PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code>
```

Justification:

The original implementation relies on multiple `if-elif` conditions, which becomes harder to maintain as the number of operations grows. Using a

mapping-based approach improves scalability, reduces code complexity, and makes it easier to add new operations while preserving the same functionality and output.

Task 5: Simplifying Search Logic in Collections

Scenario

You are reviewing legacy inventory-check code that uses manual loops to find elements.

Task Description

• Identify the explicit loop used for searching an item.

• Use AI assistance to refactor the logic into a more concise and readable form.

• Maintain the same output behavior.

Code:

```
C: > Users > Hello > 13.5.py > ...
1   #Review the given Python legacy code that uses an explicit loop and flag variable to search for an
2   # Original legacy code
3   items = ['apple', 'banana', 'cherry']
4   item_to_find = 'banana'
5   item_available = False
6   for item in items:
7       if item == item_to_find:
8           item_available = True
9           break
10  if item_available:
11      print("Item Available")
12  else:
13      print("Item Not Available")
14  # Refactored code using membership testing
15  items = ['apple', 'banana', 'cherry']
16  item_to_find = 'banana'
17  if item_to_find in items:
18      print("Item Available")
19  else:
20      print("Item Not Available")
21  # Output
22  # Item Available
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Hello\AppData\Local
\python.exe c:/Users/Hello/13.5.py
Item Available
Item Available
```

Justification:
The original code uses a manual loop and flag variable to check for item existence, which adds unnecessary complexity. Refactoring with Python's built-in collection search capabilities improves readability, reduces code length, and preserves the same output behavior.